

Technical Assignment Overview

Objective:

Build two interdependent applications—a backend API service and a frontend client—to manage and display a list of digital game items from a CSV file. The system must support user authentication, product listing with pagination, detailed product views, and a simple purchase flow that generates a receipt and stores the order.

Requirements

CSV Data Import and Database Setup

- **CSV File Details:**
 - Contains digital game items with fields such as title, description, price, etc.
 - Each item is assigned a location indicator ("JO" for Jordan or "SA" for Saudi Arabia).
- **Task:**
 - Develop a script or service to import the CSV data into a database of your choice. Provide a rationale for your chosen database (e.g., PostgreSQL, MySQL, SQLite).

Backend API Development

- **Technology:** Python (framework of your choice is acceptable; recommended options include Django, Flask, or FastAPI).
- **Endpoints to Implement:**
 - Authentication:
 - Implement a login endpoint that validates user credentials and returns an authentication token (JWT or similar).
 - Protect all other endpoints using token-based authentication.
 - Products Listing:
 - Create a paginated endpoint to list products.
 - Optionally, allow filtering by location (JO/SA).
 - Product Details:
 - Create an endpoint to retrieve detailed information for a specific product.
 - Purchase Flow:
 - Implement an endpoint to “buy” a product (only one product per request).
 - On purchase, generate an order record in the database and return order details for receipt generation.
- **Implementation Considerations:**
 - Ensure proper error handling and input validation.
 - Follow RESTful design principles.
 - Provide sensible defaults for pagination parameters (e.g., page number, page size).
 - Include API documentation to explain endpoints, expected inputs, and outputs.

Frontend Application Development

- **Technology:** Either React.js, Nextjs or Angular.
- **Pages/Components:**
 - Login Page:
 - Allow users to authenticate and securely store the authentication token.
 - Product Listing Page:
 - Use a grid system layout. (e.g., CSS Flexbox, CSS Grid, or a UI framework) to arrange the product items.
 - Style the product title distinctly (larger font size, bold, etc.) from the description and other fields.
 - Implement pagination controls that interact with the backend API.
 - Product Details Page:
 - Show detailed information about a selected product.
 - Include a “Buy” button to initiate the purchase flow.
 - Purchase/Receipt Page:
 - After a product is bought, redirect the user to a receipt page that displays order details.
- **Authentication:**
 - Ensure that every page uses the authentication token for API requests.
 - Implement redirection for unauthenticated users to the login page.

Implementation Strategy

- **Project Setup:**
 - Structure the project into two separate repositories or modules: one for the backend and one for the frontend.
 - Provide clear setup instructions in a README file.
- **CSV Import:**
 - Develop a Python script to parse the CSV and load the data into your chosen database.
 - Consider using libraries such as Python’s built-in CSV module for data processing.
- **Backend API:**
 - Select a suitable Python framework (e.g., Django REST Framework).
 - Design and implement data models for products and orders, including fields for product information, location, and order metadata.
 - Implement token-based authentication to secure the API endpoints.
 - Ensure that endpoints adhere to RESTful standards and include proper error handling and validation.
 - Document the API endpoints and provide usage instructions.
- **Frontend Application:**
 - Implement state management (using context, or Angular services) to handle authentication tokens and application state.
 - Build components for the Login, Product Listing, Product Details, and Receipt pages.
 - Use a grid system to layout product listings, ensuring that the title styling is visually distinct from other details.
 - Make sure that the pages implemented are responsive for at least two screen sizes (desktop and mobile), supporting other sizes is not required.
 - Ensure the frontend interacts correctly with the backend API, especially regarding authentication and pagination.
 - Provide clear documentation on the setup and run instructions for the frontend.
- **Authentication & Security:**
 - Implement secure token-based authentication for both the backend and frontend.
- **Documentation:**
 - Include comprehensive documentation for both applications.

- Provide a README file with detailed setup and run instructions and explaining design decisions and assumptions.

Deliverables

- **Backend Application:**
 - A CSV import script.
 - A fully functional API with endpoints for authentication, product listing (with pagination), product details, and order creation.
 - Database schema and migration scripts (if applicable).
 - Documentation outlining API endpoints and usage.
- **Frontend Application:**
 - A login page implementing token-based authentication.
 - A product listing page utilizing a grid layout with distinct styling for product titles.
 - A product details page with a “Buy” functionality.
 - A purchase/receipt page that displays order details.
 - Setup and run documentation.
- **General Documentation:**
 - A README file with instructions for setting up and running both the backend and frontend.
 - Inline code documentation and comments detailing key design choices and assumptions.